Anuprita Mhapankar **D20A** **34**

Blockchain Experiment 10

AIM: Explore the Cryptocurrency Landscape

THEORY:

Cryptocurrencies are decentralized digital assets built on blockchain technology, enabling secure, transparent, and immutable transactions without the need for intermediaries. Among various types of cryptocurrencies, **stablecoins** are designed to maintain a stable value by being pegged to fiat currency such as the US Dollar.

Stablecoins are a type of cryptocurrency designed to maintain a stable value by being pegged to an underlying asset such as a fiat currency (e.g., US Dollar), cryptocurrency, or algorithmic mechanism. Unlike volatile cryptocurrencies like Bitcoin, stablecoins aim to reduce price fluctuations, making them suitable for trading, payments, and decentralized finance (DeFi) applications.

Fiat-backed stablecoins are supported by real-world currencies like USD, EUR, etc. For every coin issued, an equivalent amount of money is stored in reserves (like banks).
Example: USDT (Tether), USDC, TUSD

Crypto-backed stablecoins are backed by other cryptocurrencies (like ETH). Since crypto prices fluctuate, extra collateral is required to maintain stability.

Example: DAI

Hybrid stablecoins use a combination of collateral (like USDC) and algorithmic mechanisms to maintain price stability.

Example: FRAX

This experiment focuses on analyzing stablecoins including USDT, USDC, DAI, FRAX, and TUSD using real-time data collected from multiple sources:

- **CoinGecko API:** Provided historical price, trading volume, and market capitalization data
- **CoinMarketCap / Crypto platforms:** Used for validation and market insights
- **Online resources:** Used for understanding adoption and use cases

Cryptocurrency Landscape:

- **Stablecoins** – Maintain stable value (USDT, USDC, DAI)
- **Store of Value** – Bitcoin
- **Utility Tokens** – Ethereum
- **Governance Tokens** – UNI, AAVE

Key Characteristics of Stablecoins:

- Low volatility compared to other cryptocurrencies
- Used for trading, payments, and DeFi applications
- Backed by fiat reserves, crypto assets, or hybrid mechanisms

TECHNOLOGICAL ANALYSIS:

Recent advancements in blockchain technology include:

- Consensus Mechanisms: PoW, PoS for secure validation
- Scalability: Layer-2 solutions like rollups
- Interoperability: Cross-chain communication
- Privacy: Zero-knowledge proofs

Stablecoins leverage these technologies to provide efficient and scalable financial solutions in decentralized ecosystems.

MARKET TRENDS & ADOPTION:

- Increasing use of stablecoins in DeFi platforms
- Growing merchant acceptance for payments
- Rising number of crypto wallets and users
- Integration with traditional financial systems
- Regulatory developments across countries

TASK PERFORMED:

1. Collected historical price and volume data using CoinGecko API
2. Processed data using Python libraries (Pandas, Requests)
3. Visualized data using Matplotlib and Seaborn
4. Compared price stability, volatility, and market dominance
5. Classified stablecoins based on type and use case

CODE:

```
!pip install requests pandas matplotlib seaborn -q

import requests
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import time
STABLECOINS = {
    'tether':          'USDT',
    'usd-coin':        'USDC',
```

```
    'dai':              'DAI',
    'frax':             'FRAX',
    'true-usd':         'TUSD'
}
DAYS  = 90        # last 90 days
COLOR = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']
def fetch_market_chart(coin_id, days=DAYS, retries=3):
    url = f"https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart"
    params = {"vs_currency": "usd", "days": days, "interval": "daily"}

    for attempt in range(retries):
        try:
            r = requests.get(url, params=params, timeout=20)
            if r.status_code == 200:
                return r.json()
            elif r.status_code == 429:
                wait = 15 * (attempt + 1)    # 15s → 30s → 45s
                print(f"  ⏳ Rate limited. Waiting {wait}s before retry
{attempt+1}/{retries}...")
                time.sleep(wait)
            else:
                print(f"  ⚠ Could not fetch {coin_id}: HTTP
{r.status_code}")
                return None
        except Exception as e:
            print(f"  Error fetching {coin_id}: {e}")
            return None

    print(f"  Failed after {retries} retries: {coin_id}")
    return None

# --- Fetch historical data for each stablecoin ---
price_frames = {}
volume_frames = {}

print("Fetching data from CoinGecko API (with rate-limit handling)...\n")
for coin_id, ticker in STABLECOINS.items():
    print(f"  → Fetching {ticker} ({coin_id})...")
    data = fetch_market_chart(coin_id)
    if data:
```

```
        pdf = pd.DataFrame(data['prices'], columns=['ts', 'price'])
        vdf = pd.DataFrame(data['total_volumes'], columns=['ts',
'volume'])
        pdf['date'] = pd.to_datetime(pdf['ts'], unit='ms').dt.normalize()
        vdf['date'] = pd.to_datetime(vdf['ts'], unit='ms').dt.normalize()
        pdf = pdf.drop_duplicates('date').set_index('date')
        vdf = vdf.drop_duplicates('date').set_index('date')
        price_frames[ticker] = pdf['price']
        volume_frames[ticker] = vdf['volume']
        print(f" {ticker} fetched successfully!")

    time.sleep(8)   # ← increased from 1.2s to 8s between each call

prices_df  = pd.DataFrame(price_frames)
volumes_df = pd.DataFrame(volume_frames)
print("\n All data fetched!")
def fetch_current_data():
    """Fetch current market cap, volume, price for all stablecoins."""
    ids = ",".join(STABLECOINS.keys())
    url = "https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd"
    params = {
        "vs_currency": "usd",
        "ids": ids,
        "order": "market_cap_desc",
        "per_page": 10,
        "page": 1,
        "sparkline": False
    }
    r = requests.get(url, params=params, timeout=15)
    return pd.DataFrame(r.json())
# Current snapshot
current_df = fetch_current_data()
print("\n Data collection complete!\n")
print(current_df[['name','current_price','market_cap','total_volume']].to_
string(index=False))
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Define colors
```

```
COLOR = ['blue', 'orange', 'green', 'red', 'purple']
sns.set_theme(style="darkgrid")

fig = plt.figure(figsize=(12, 6))

tickers = list(STABLECOINS.values())
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in prices_df.columns:
        plt.plot(prices_df.index, prices_df[ticker], label=ticker)

plt.axhline(1.0, linestyle='--', label='$1 Peg')

plt.title('Price Stability vs $1 Peg')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in prices_df.columns:
        deviation = (prices_df[ticker] - 1.0) * 100
        plt.plot(prices_df.index, deviation, label=ticker)

plt.axhline(0, linestyle='--')

plt.title('Deviation from $1 Peg (%)')
plt.xlabel('Date')
plt.ylabel('Deviation (%)')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(8,5))

if not current_df.empty:
```

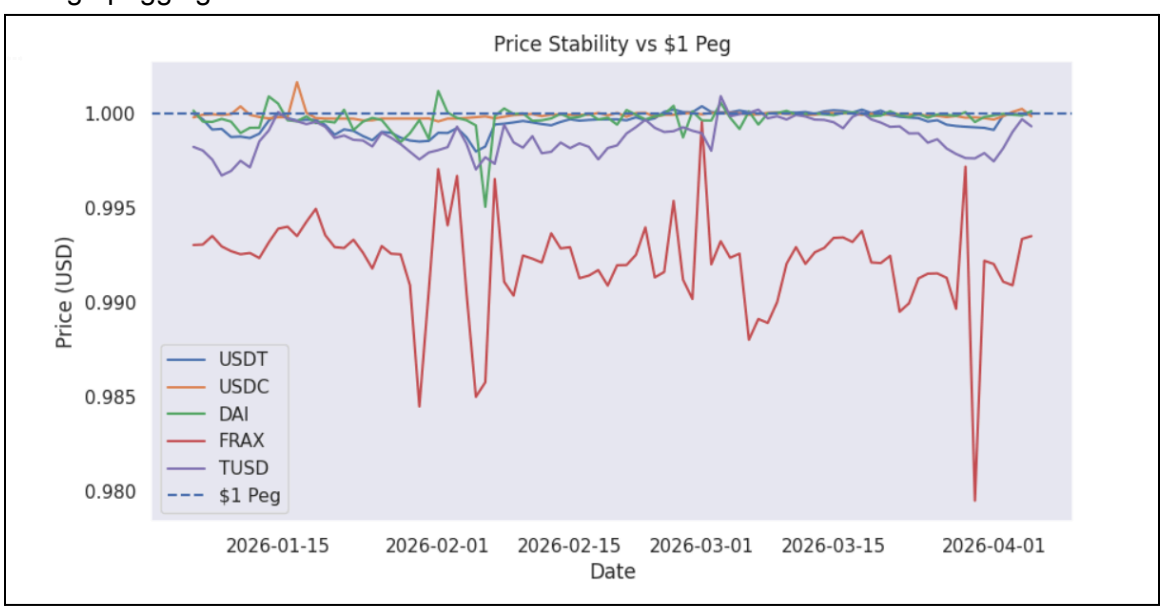
```
    names = current_df['symbol'].str.upper()
    mcaps = current_df['market_cap'] / 1e9

    plt.bar(names, mcaps)

    for i, val in enumerate(mcaps):
        plt.text(i, val, f'{val:.1f}B', ha='center')

plt.title('Market Capitalization (Billion USD)')
plt.xlabel('Stablecoin')
plt.ylabel('Market Cap (Billion USD)')
plt.grid()

plt.show()
plt.figure(figsize=(6,6))

if not current_df.empty:
    labels = current_df['symbol'].str.upper()
    sizes = current_df['market_cap']

    plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)

plt.title('Market Cap Dominance')
plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
    if ticker in volumes_df.columns:
        plt.plot(volumes_df.index, volumes_df[ticker] / 1e9, label=ticker)

plt.title('Trading Volume (Billion USD)')
plt.xlabel('Date')
plt.ylabel('Volume')
plt.legend()
plt.grid()

plt.show()
plt.figure(figsize=(10,5))

for ticker in tickers:
```

```
    if ticker in prices_df.columns:
        returns = prices_df[ticker].pct_change()
        volatility = returns.rolling(7).std() * 100
        plt.plot(prices_df.index, volatility, label=ticker)

plt.title('7-Day Volatility (%)')
plt.xlabel('Date')
plt.ylabel('Volatility')
plt.legend()
plt.grid()

plt.show()
classification = {
    'Coin': ['USDT', 'USDC', 'DAI', 'FRAX', 'TUSD'],
    'Type': ['Fiat-backed', 'Fiat-backed', 'Crypto-backed',
             'Hybrid', 'Fiat-backed'],
    'Use Case': ['Trading & Liquidity', 'Payments & DeFi', 'DeFi Lending',
                 'Stable DeFi Ecosystem', 'Cross-border Payments']
}
cls_df = pd.DataFrame(classification)
print(cls_df.to_string(index=False))
```

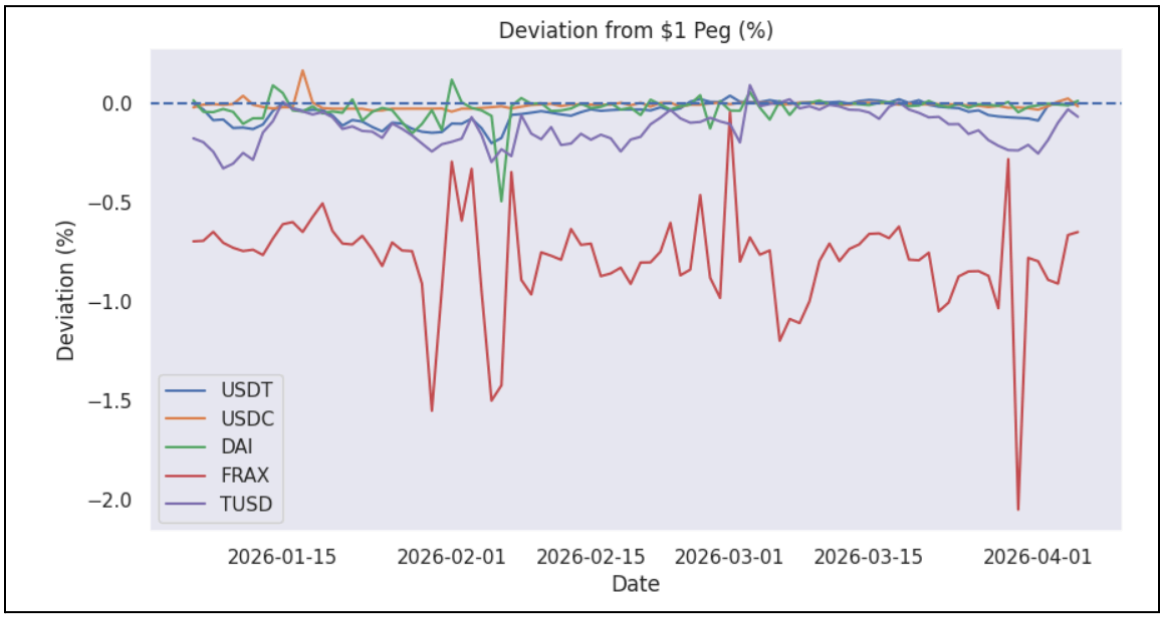
OUTPUT:

1. **Price Stability**

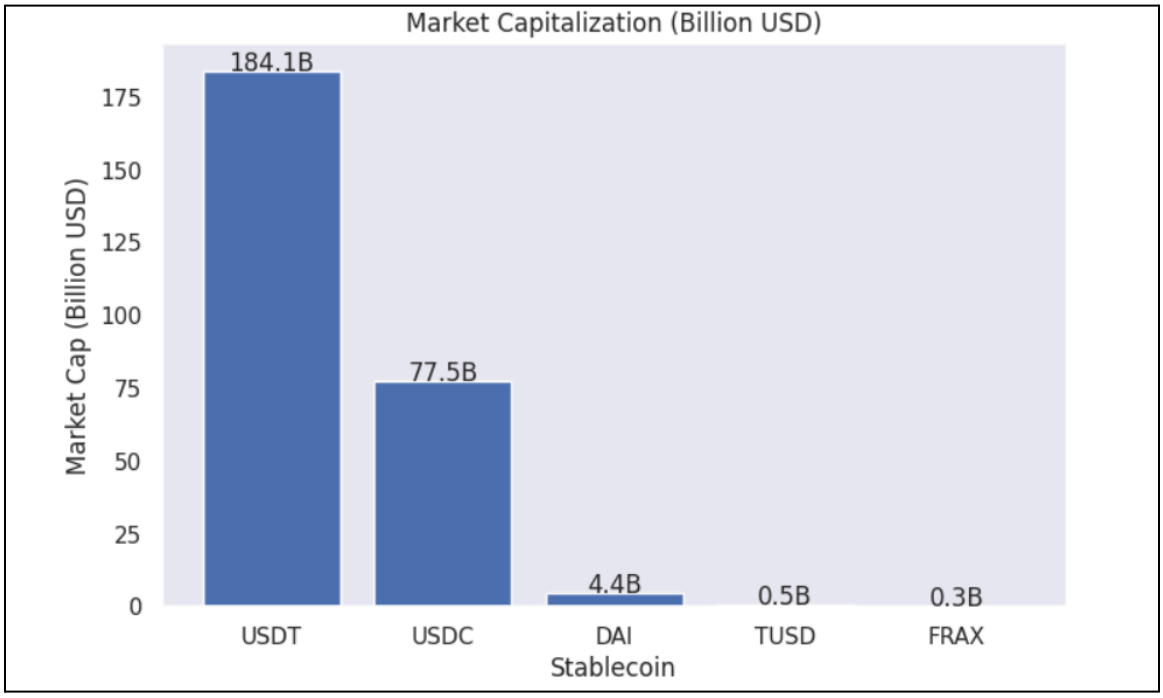
   This graph shows the price movement of stablecoins over time. All selected stablecoins remain close to the $1 value, demonstrating their ability to maintain price stability through pegging mechanisms.



2. **Deviation from $1 Peg**
   This graph represents the percentage deviation of stablecoin prices from the $1 peg. The minimal fluctuations indicate low de-pegging risk and high reliability of stablecoins in maintaining consistent value.

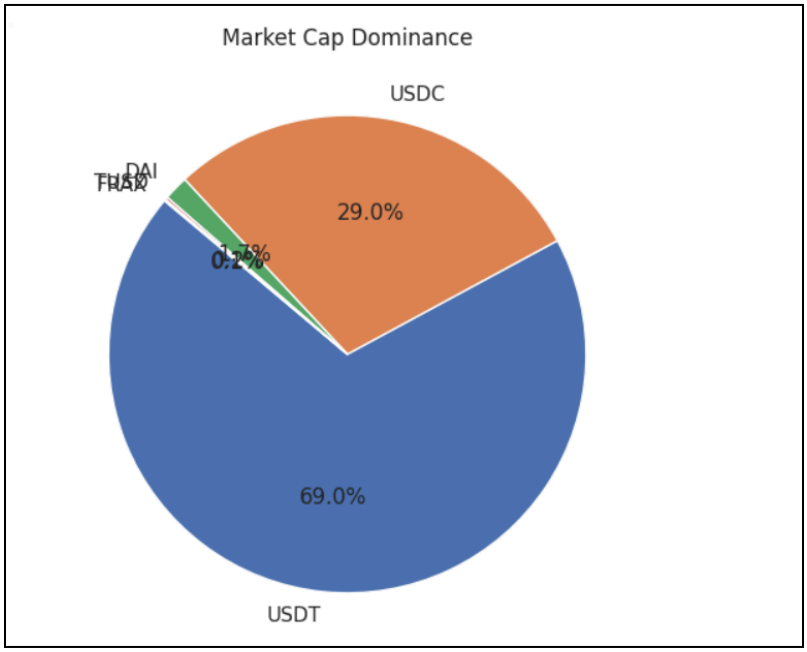


3. **Market Capitalization**
   This graph compares the market capitalization of different stablecoins. It shows that USDT and USDC dominate the market, indicating higher adoption, liquidity, and trust among users.



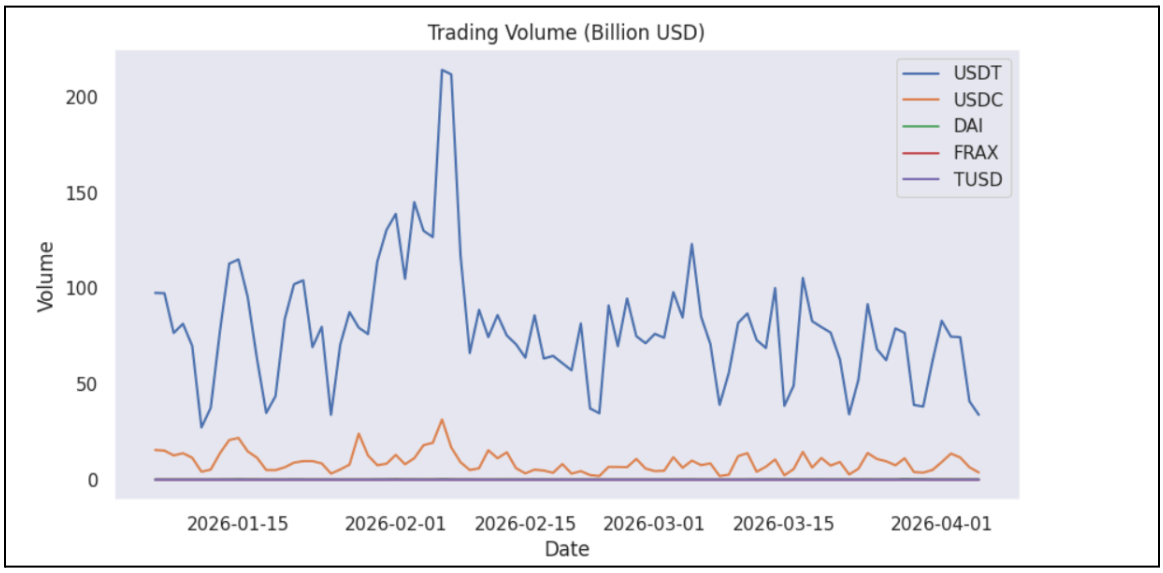
4. **Market Dominance**

This pie chart illustrates the proportion of total market share held by each stablecoin. It highlights that a few major stablecoins control a large portion of the market.

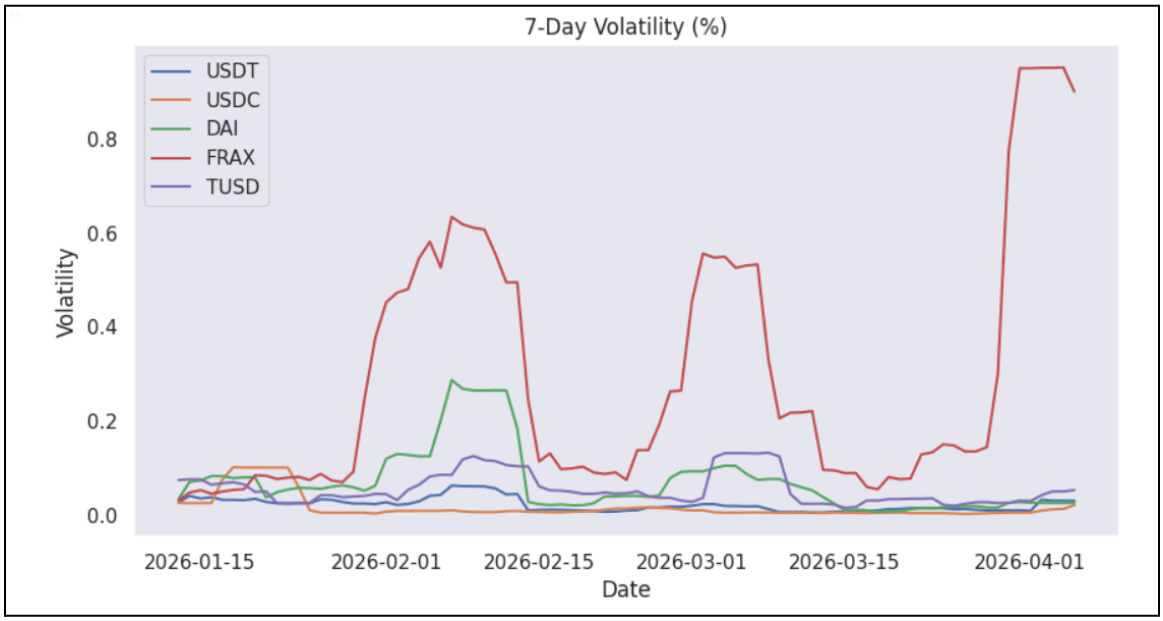


5. **Trading Volume**

This graph shows the daily trading volume of stablecoins over time. Higher trading volume reflects greater usage in transactions, exchanges, and DeFi applications.



6. **7-Day Volatility**
   This graph represents the rolling volatility of stablecoin prices. The low volatility confirms that stablecoins experience minimal price fluctuations compared to other cryptocurrencies.



The selected cryptocurrencies are classified based on their backing mechanism and use cases.

Coin	Type	Use Case
USDT	Fiat-backed	Trading & Liquidity
USDC	Fiat-backed	Payments & DeFi
DAI	Crypto-backed	DeFi Lending
FRAX	Hybrid	Stable DeFi Ecosystem
TUSD	Fiat-backed	Cross-border Payments

CONCLUSION:

The experiment successfully explored the cryptocurrency landscape by analyzing stablecoins using real-time data. The results demonstrated that stablecoins maintain price stability, exhibit low volatility, and play a crucial role in trading, payments, and decentralized finance. This study highlights the importance of blockchain technology in real-world financial applications.